

Constraint-Based Problem Solving

Given Problem

There are 5 courses:

- **A** – Artificial Intelligence
- **B** – Database Systems
- **C** – Computer Networks
- **D** – Operating Systems
- **E** – Software Engineering

There are 3 examination slots: $\{Slot1, Slot2, Slot3\}$

The conflicting pairs that cannot be in the same slot are:

$$A - B, A - C, B - C, B - D, C - D, C - E, D - E$$

The objective is to create a timetable in which no conflicting courses are scheduled in the same slot. These are the exact constraints given in the assessment

(a) PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

P – Performance Measure The

scheduling agent should:

1. Schedule all 5 courses.
2. Use only the 3 available slots.
3. Ensure that no conflicting courses are placed in the same slot.
4. Produce a valid and conflict-free examination timetable.

E – Environment

The environment consists of:

- 5 university courses: A, B, C, D and E.
- 3 examination slots: Slot1, Slot2 and Slot3.
- Student enrollment conflicts between specific pairs of courses.
- One examination slot available per day.

A – Actuators The

agent can:

- Assign a course to Slot1.
- Assign a course to Slot2.
- Assign a course to Slot3.
- Change an assignment when a conflict occurs.

S – Sensors

The agent can check:

- The current course assignment.
- The available slots.
- Whether two conflicting courses are assigned to the same slot.
- Whether all courses have been successfully scheduled.

PEAS Summary

Component	Description
Performance	Produce a complete conflict-free timetable
Environment	5 courses, 3 slots and enrollment conflicts

Actuators	Assign courses to examination slots
Sensors	Detect assignments, conflicts and available slots

Conclusion: The scheduling agent's main objective is to assign every course to a slot while satisfying all the given conflict constraints.

(b) Type of Intelligent Agent

Goal-Based Agent

A Goal-Based Agent is the most suitable for this scheduling problem.

The main goal is:

Create a complete examination timetable in which no conflicting courses are assigned to the same slot.

The agent has to consider different possible assignments and select assignments that satisfy the goal.

Justification

- A Simple Reflex Agent only reacts to the current situation and is not suitable for systematically constructing the complete timetable.
- A Model-Based Agent can maintain information about previous states, but the important requirement here is achieving a specific valid timetable.
- A Goal-Based Agent can evaluate assignments according to whether they help achieve the conflict-free timetable.
- A Utility-Based Agent would be more suitable if the problem included preferences such as minimizing student inconvenience, balancing exams, or minimizing gaps between exams.

Therefore:

Goal-Based Agent is the most appropriate choice because the scheduling task has a clearly defined goal: find a conflict-free timetable.

(c) CSP Formulation + Constraint Graph

A Constraint Satisfaction Problem (CSP) consists of:

1. Variables
2. Domains
3. Constraints

1. Variables

Each course is a variable: X

$= \{A, B, C, D, E\}$

where:

- A = Artificial Intelligence
- B = Database Systems
- C = Computer Networks
- D = Operating Systems
- E = Software Engineering

2. Domains

Each course can be assigned to one of three slots:

$$\begin{aligned} D(A) &= D(B) = D(C) = D(D) = D(E) \\ &= \{Slot1, Slot2, Slot3\} \end{aligned}$$

3. Constraints

Conflicting courses must have different slots.

$$A \neq B$$

$$A \neq C$$

$$B \neq C$$

$$B \neq D$$

$$C \neq D$$

$$C \neq E$$

$$\begin{array}{c} D \\ \neq E \end{array}$$

Therefore, the complete constraint set is:

$$\{A \neq B, A \neq C, B \neq C, B \neq D, C \neq D, C \neq E, D \neq E\}$$

Constraint Graph

Vertices

$$V = \{A, B, C, D, E\}$$

Edges

$$E = \{AB, AC, BC, BD, CD, CE, DE\}$$

The graph can be represented as:



More specifically, the edges are:

- A — B
- A — C
- B — C
- B — D
- C — D
- C — E
- D — E

An edge means that the two connected courses must be assigned different slots.

(d) Plain Backtracking Search Program:

```
courses = ['A', 'B', 'C', 'D', 'E']
slots = ['Slot1', 'Slot2', 'Slot3']
conflicts = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'C'),
    ('B', 'D'),
    ('C', 'D'),
    ('C', 'E'),
    ('D', 'E')
]
assignment = {}
def is_valid(course, slot):
    for c1, c2 in conflicts:
        if course == c1 and c2 in assignment:
            if assignment[c2] == slot:
                return False
        if course == c2 and c1 in assignment:
            if assignment[c1] == slot:
                return False
    return True
def backtrack():
    if len(assignment) == len(courses):
        return True
    course = courses[len(assignment)]
    for slot in slots:
        print("Trying:", course, "=", slot)
        if is_valid(course, slot):
            assignment[course] = slot
            print("Accepted:", course, "=", slot)
            if backtrack():
                return True
            print("Backtracking:", course, "=", slot)
            del assignment[course]
        else:
            print("Rejected:", course, "=", slot)
    return False
if backtrack():
    print("\nFinal Valid Timetable")
    print("-----")
    for course in courses:
        print(course, "->", assignment[course])
else:
    print("No solution found")
```

Output:

```
Trying: A = Slot1
Accepted: A = Slot1
Trying: B = Slot1
Rejected: B = Slot1
Trying: B = Slot2
Accepted: B = Slot2
Trying: C = Slot1
Rejected: C = Slot1
Trying: C = Slot2
Rejected: C = Slot2
Trying: C = Slot3
Accepted: C = Slot3
Trying: D = Slot1
Accepted: D = Slot1
Trying: E = Slot1
Rejected: E = Slot1
Trying: E = Slot2
Accepted: E = Slot2

Final Valid Timetable
-----
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2
```

(e) Backtracking with Forward Checking Program:

```
courses = ['A', 'B', 'C', 'D', 'E']
slots = ['Slot1', 'Slot2', 'Slot3']
conflicts = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'C'),
    ('B', 'D'),
    ('C', 'D'),
    ('C', 'E'),
    ('D', 'E')
]
domains = {
    'A': ['Slot1', 'Slot2', 'Slot3'],
    'B': ['Slot1', 'Slot2', 'Slot3'],
    'C': ['Slot1', 'Slot2', 'Slot3'],
    'D': ['Slot1', 'Slot2', 'Slot3'],
    'E': ['Slot1', 'Slot2', 'Slot3']
}
assignment = {}
def get_neighbors(course):
    neighbors = []
    for c1, c2 in conflicts:
        if course == c1:
            neighbors.append(c2)
        elif course == c2:
            neighbors.append(c1)
    return neighbors
def backtracking():
    if len(assignment) == len(courses):
        return True
    course = courses[len(assignment)]
    for slot in domains[course][:]:
        valid = True
        for neighbor in get_neighbors(course):
            if neighbor in assignment:
                if assignment[neighbor] == slot:
                    valid = False
                    break
        if not valid:
            print("Rejected:", course, "=", slot)
            continue
        print("\nTrying:", course, "=", slot)
        old_domains = {
            c: domains[c][:]
            for c in courses
        }
        assignment[course] = slot
        failure = False
        for neighbor in get_neighbors(course):
            if neighbor not in assignment:
                if slot in domains[neighbor]:
                    domains[neighbor].remove(slot)
                if len(domains[neighbor]) == 0:
                    failure = True
                    break
        print("Domains after", course, "=", slot)
        for c in courses:
            if c not in assignment:
                print(c, ":", domains[c])
        if not failure:
            if backtracking():
                return True
        print("Backtracking from:", course, "=", slot)
        assignment.pop()
        for c in courses:
            domains[c] = old_domains[c]
    return False
if backtracking():
    print("\nFinal Valid Timetable")
    print("-----")
    for course in courses:
        print(course, "->", assignment[course])
else:
    print("No solution found")
```

Output:

```
Trying: A = Slot1
Domains after A = Slot1
B : ['Slot2', 'Slot3']
C : ['Slot2', 'Slot3']
D : ['Slot1', 'Slot2', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']

Trying: B = Slot2
Domains after B = Slot2
C : ['Slot3']
D : ['Slot1', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']

Trying: C = Slot3
Domains after C = Slot3
D : ['Slot1']
E : ['Slot1', 'Slot2']

Trying: D = Slot1
Domains after D = Slot1
E : ['Slot2']

Trying: E = Slot2
Domains after E = Slot2

Final Valid Timetable
-----
A -> Slot1
B -> Slot2
C -> Slot3
D -> Slot1
E -> Slot2
```

(f) Comparison

Feature	Plain Backtracking	Forward Checking
Variable order	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$
Value order	$S1 \rightarrow S2 \rightarrow S3$	$S1 \rightarrow S2 \rightarrow S3$
Assignment attempts	9	5
Failed attempts	4	0
Backtracks	0 actual returns	0
Domain pruning	No	Yes
Search effort	Higher	Lower
Efficiency	Lower	Higher

In ordinary backtracking, the algorithm may try an assignment even when it will create a conflict later.

For example, after:

$$A = S1$$

plain backtracking still tries: $B = S1$

and discovers the conflict only after making the attempt.

Forward checking immediately removes **S1** from B's domain:

$$B = \{S2, S3\}$$

So the invalid assignment is **never attempted**.

Similarly, after: $A = S1, B = S2$

forward checking knows that C cannot use S1 or S2:

$$C = \{S3\}$$

Therefore, C can be assigned directly to S3 without trying invalid values.

Final Analysis

Plain backtracking required:

9 attempts

while forward checking required:

5 attempts

So forward checking reduced the number of assignment attempts by:

$$9 - 5 = 4$$

4 fewer attempts

Percentage reduction:

$$9-5/9*100=44.44\%$$

Thus, forward checking reduced the assignment attempts by approximately:

44.44%

Conclusion

The CSP has 5 variables, each with the domain: $\{Slot1, Slot2, Slot3\}$

and 7 binary constraints:

$$A - B, A - C, B - C, B - D, C - D, C - E, D - E$$

Both methods produce the valid timetable:

Course	Examination Slot
A – Artificial Intelligence	Slot1
B – Database Systems	Slot2
C – Computer Networks	Slot3
D – Operating Systems	Slot1
E – Software Engineering	Slot2

[A=S1, B=S2, C=S3, D=S1, E=S2]

The Forward Checking approach is more efficient because it prunes inconsistent values immediately, reducing unnecessary assignment attempts from 9 to 5 while satisfying all constraints.